ROS2 Fundamentals

Madhura Korgaonkar
Department of Computer Science and Engineering, MIT, Manipal, Karnataka, India

***Abstract*—This report presents a study and implementation-oriented overview of the ROS 2 Humble communication model and beginner client-library workflow. It covers the ROS 2 computation graph, nodes, topics, services, actions, parameters, custom interfaces, workspaces, colcon, and Quality of Service (QoS). Two solution graphs are included: a publisher/subscriber system using the /chatter topic and a client/server system using a custom AddThreeInts service interface. The report also explains when topics and services should be selected, how interfaces are generated, how a workspace is organized and built, and how QoS policies affect communication reliability and timing. The discussion follows the structure and terminology of the ROS 2 Humble beginner tutorials and official documentation.**

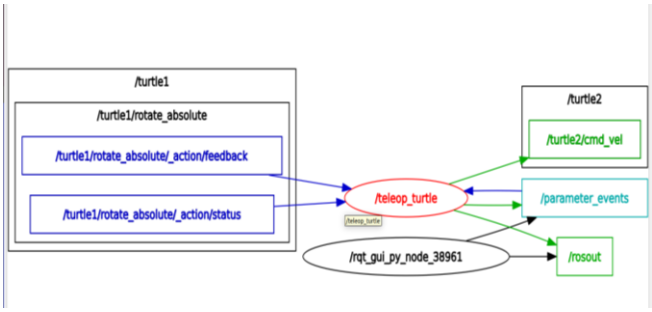
Keywords— ROS 2, Humble, computation graph, nodes, topics, services, actions, parameters, custom interfaces, colcon, workspace, QoS

I. INTRODUCTION

Robot Operating System 2 or ROS 2 is a middleware framework for building distributed robotic applications. Instead of requiring one large program to perform sensing, planning, control, visualization, and diagnostics, ROS 2 allows a system to be decomposed into communicating nodes. The default ROS 2 communication stack is based on DDS concepts and provides discovery and configurable Quality of Service policies. This makes the same programming model suitable for processes on one computer as well as distributed robotic systems. The beginner workflow studied for this report begins with command-line tools and environment configuration, followed by turtlesim and rqt, nodes, topics, services, parameters, actions, logging, and launch files. The client-library part then introduces colcon, workspaces, packages, publisher/subscriber programs, and custom interfaces. The objective is not only to run individual commands, but to understand how each component contributes to a complete computation graph.

II. THE ROS 2 COMPUTATION GRAPH

The ROS graph is the network of ROS 2 elements that data together. Its vertices are nodes, and its edges are the topics, services, actions and parameters through which nodes communicate. There is no central master: nodes find each other at run time through the discovery mechanism of the DDS middleware. Nodes discover each other only if they share the same ROS_DOMAIN_ID (0 by default), and setting ROS_LOCALHOST_ONLY=1 confines traffic to on machine. Nodes are the computational participants; topics provide asynchronous data streams; services provide request/response communication; actions provide long-running goal-oriented interactions; and parameters store configurable node state. Discovery is handled through the ROS 2 middleware rather than through a single centralized master, allowing nodes to find compatible communication endpoints on the network. The graph can therefore be viewed at several levels. At the node level, the graph identifies which processes exist. At the topic level, it shows publishers and subscribers. At the service and action levels, it shows clients and servers. Tools such as ros2 node list, ros2 topic list, ros2 service list, ros2 action list, and rqt_graph provide different views of this structure.



III. CORE ROS 2 ENTITIES

A. Nodes

A node is a single-purpose executable responsible for one logical task, such as reading a sensor or driving a motor. A node is started with ros2 run <package> <executable> and inspected with ros2 node list and ros2 node info node_name, which report its publishers, subscribers, service servers and clients, and action servers and clients. Remapping allows several instances of one executable to run side by side.

Remapping :-

ros2 run turtlesim turtlesim_node --ros-args --remap __node:=my_turtle

B. Topics

A topic is a named channel for asynchronous publish/subscribe communication. A publisher sends messages without addressing a particular subscriber, while one or more subscribers receive messages of the compatible message type. Topics are therefore appropriate for continuous or streaming data such as sensor readings, robot pose, camera frames, velocity commands, or detected objects. Topics provide anonymous, asynchronous, many-to-many publish–subscribe communication. Any number of publishers and subscribers can use a topic, and every message on it has one fixed type, for example, geometry_msgs/msg/Twist.

The command “ros2 topic list –t", lists the topic types along with names and for more detailed information about a topic the --verbose or –v flag can be used.

The command “ros2 interface show <msg_type>” can be used to check the the specific data structure the message expects.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

And then to publish the data to a topic the command “ros2 topic pub <topic_name> <msg_type> '<args>'” is used. To publish the command just once “--once" option is added.

To view the rate at which the topic is being published, “ros2 topic hz /turtle1/pose” is used.

To check the bandwidth used by a topic, the command, “ros2 topic bw /turtle1/pose” is used.

To find the topics having the same type, the command, “ros2 topic find <topic_type>”, can be used.

you can publish data to a topic directly from the command line using: ros2 topic pub <topic_name> <msg_type> '<args>'

C. Services

A service follows a call-and-response model: a client sends one request and the server returns one response. A service type therefore has a request part and a response part, and each service name has a single server but may have many clients.

This interaction is appropriate for short operations where the caller expects a defined result, such as resetting a simulator, changing a mode, querying a state, or triggering a simple computation.

Services are listed with ros2 service list and this is given the argument “–t”, to list the type as well. Turtlesim offers /spawn, /kill, /clear, /reset and /turtle1/set_pen, among others, and every node also exposes services for reading and changing its parameters.

To check the arguments needed to call a particular service the ros2 interface show command is used and then the service can bt called using ros2 service call. The ros2 service type and ros2 service find can be used to find the type of a known service and find a service when its type is known.

D. Parameters

Parameters are configuration values stored in a node, such as integers, floating-point numbers, booleans, strings and arrays, that can be read and modified while it runs. ros2 param list and get are used to access, set is used to modify and ros2 param dump and load can save and restore a node’s settings in a YAML file, which can also be supplied at start-up with --params-file to start a new node with the modified parameters. Changing turtlesim’s background_r, background_g and background_b parameters recolours its canvas without a restart. Many other behaviours such as timer period, threshold, frame name, or controller setting can be changed without rewriting the program.

E. Actions

Actions serve long-running tasks. A client sends a goal, the server accepts it, reports feedback while it works and finally returns a result, and the client may cancel the goal at any time. Internally an action is built from three services (send goal, cancel goal, get result) and two topics (feedback and status). Turtlesim’s /turtle1/rotate_absolute action, of type turtlesim/action/RotateAbsolute, can be driven with ros2 action send_goal --feedback, while ros2 action list –t and ros2 action info show the available actions.

F. TOPICS VERSUS SERVICES

The main distinction is the communication pattern. A topic is a stream-oriented publish/subscribe mechanism, whereas a service is a request/response mechanism. A publisher does not need to know which subscribers will consume its messages, while a service client explicitly asks a service server to perform an operation and return a response.

A topic is the right choice when data is produced continuously, or whenever a quantity changes and several consumers may need it, for example IMU, LiDAR or wheel-speed measurements, where a lost sample is soon replaced by a newer one.

A service is the right choice for an occasional operation that needs a definite reply, such as resetting odometry, spawning a turtle or asking whether a value is valid.

Aspect	Topic	Service
Pattern	Publish–subscribe	Request–response
Cardinality	Many publishers, many subscribers	One server, many clients
Timing	Asynchronous, continuous stream	On demand; client waits for the reply
Reply	None	Response for every request
Typical data	Sensor readings, pose, commands	Configuration, queries, one-shot triggers
Example	/turtle1/cmd_vel	/spawn, /clear

IV. CUSTOM INTERFACES

ROS 2 describes the data exchanged between nodes in interface files: .msg files for topics, .srv files for services, in which the request and the response are separated by a line of three dashes, and .action files for actions. A .msg file has one field per line in the form type name, where the type is a built-in type (bool, byte, char, float32, float64, int8 to int64, uint8 to uint64 or string), an array of one, or another message. Interface generation is configured in an ament_cmake package, so interfaces are normally kept in a separate *_interfaces package on which the node packages depend. Such a package needs the following in CMakeLists.txt:

```
find_package(rosidl_default_generators REQUIRED)
rosidl_generate_interfaces(${PROJECT_NAME}
"msg/Name.msg")
```

together with the buildtool_depend, exec_depend and member_of_group entries in package.xml.

After building and sourcing the workspace, ros2 interface show displays the generated type.

V. COLCON AND WORKSPACE STRUCTURE

A workspace is a directory that holds ROS 2 packages and is built with colcon. The workspace used here has the layout below: src/ holds the packages written by the user, build/ the intermediate build files, install/ the installed

executables, generated interface code and setup scripts, and log/ the build logs.

Colcon build compiles the packages in dependency order; --packages-select restricts the build to chosen packages, and --symlink-install lets edits to Python files take effect without rebuilding. Environments are layered: the ROS 2 installation in /opt/ros/humble is the underlay, and the workspace is an overlay activated by running source install/setup.bash in every new terminal after the underlay has been sourced. Every package contains a package.xml listing its name, version and dependencies. A C++ or interface package (ament_cmake) adds a CMakeLists.txt, while a Python package (ament_python) uses setup.py and setup.cfg, with each node registered under console_scripts so that ros2 run can find it. Packages are created with ros2 pkg create --build-type <packages> <name>.<type> --dependencies

VI. QUALITY OF SERVICE

Quality of Service (QoS) policies let a publisher and a subscriber agree on how data is delivered, which matters on lossy wireless links and for nodes that join late. A subscription requests the minimum quality it will accept and a publisher offers the maximum it can provide, and the two connect only when the offer meets the request. For example, a reliable subscriber cannot receive from a best-effort publisher, and a transient-local subscriber cannot receive from a volatile publisher; the opposite combinations are allowed.

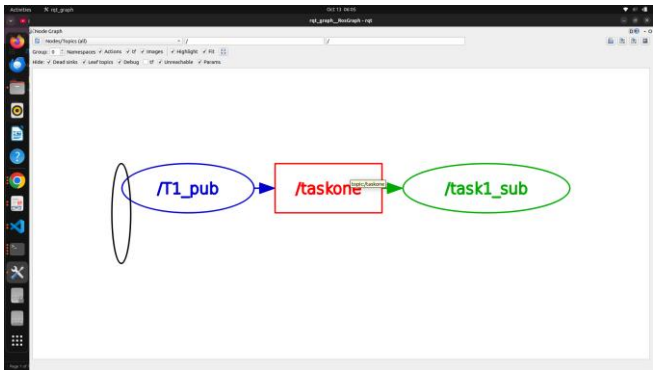
VII. IMPLEMENTATION

A. Q1: Wheel Speed From a Custom Message

The my_robot_interfaces package defines TaskOne.msg with two float64 fields:
float64 ang_vel
float64 radius

The string travels from node1 to node2, from node2 to node3, and the verdict returns from node3 to node2.

The publisher node task1_pub repeatedly asks the user for the angular velocity in rad/s and the wheel radius in m, rejects non-numeric input, and publishes each pair as a TaskOne message on /taskone. The subscriber node task1_sub subscribes to /taskone; for every message, it computes the longitudinal speed of the wheel, assuming rolling without slip, and prints it in m/s: $v = \omega r$, where ω is ang_vel and r is radius.



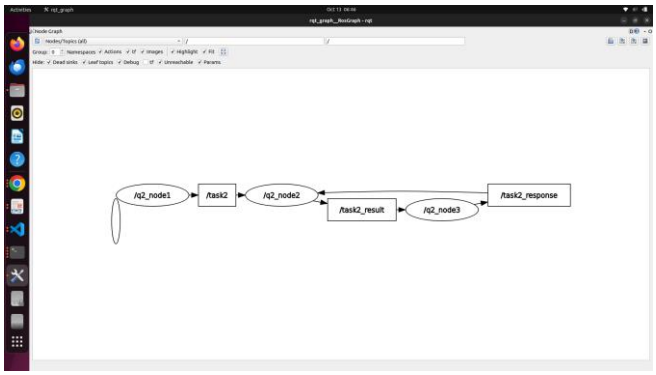
B. Q2: Palindrome Pipeline With Three ROS 2 Nodes

The palindrome pipeline consists of three ROS 2 nodes: q2_node1, q2_node2, and q2_node3. Node 1 accepts a string from the user and publishes it on /task2 using the custom RT2String message. A timer publishes the string every second, while a background thread allows the user to change the input.

Node 2 subscribes to /task2 and checks whether the string is a palindrome, ignoring case. It publishes 1 if the string is a palindrome and 0 otherwise, using std_msgs/Int8 on /palindrome_flag.

Node 3 subscribes to /palindrome_flag and publishes "Yes" or "No" using std_msgs/String on /verdict. Node 2 receives the verdict and displays it alongside the corresponding string.

The three topics provide the complete communication loop, which can be visualized using rqt_graph.



I. CONCLUSION

Working through the ROS 2 Humble tutorials gave a consistent picture of how a robot application is decomposed into nodes joined by topics, services, actions and parameters, how interfaces and QoS settings shape that communication, and how colcon packages and workspaces organize the code.

this is a level 4 heading: It's recommended to write your text in a separate document and then add it to this template once it's complete. When copying text into the template from another document, make sure that the is another level 4 heading: It's also possible to add bullet points when appropriate, using the "bullet list" style: Insert a table

or figure after the point where it is first cited in the text.This will automatically number the table for you. Any footnotes appear below the table, using the “table footnote” style. Footnotes are indicated by superscript lowercase letters within the table. An example of a table can be seen in Table I, below.Heading for a TableYou can cite your references in text by including the corresponding number, in square brackets [1]. If you need to cite a specific part of the source, you can include a page number [2, p. 13] or range [3, pp. 41–56].

ACKNOWLEDGMENTS

I would like to thank the Formula Manipal team for providing this opportunity to learn and work with ROS 2. This task helped me develop practical knowledge of ROS 2 nodes, topics, custom messages, publishers, subscribers, and inter-node communication.

REFERENCES

[1] Open Robotics, “Installing ROS 2 Humble on Ubuntu,” *ROS 2 Documentation*. [Online]. Available: https://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debs.html

[2] Open Robotics, “Beginner: CLI Tools,” *ROS 2 Documentation*. [Online]. Available: https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools.html

[1] Open Robotics, “Beginner: Client Libraries,” *ROS 2 Documentation*. [Online]. Available: https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries.html